

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Pneumonia Detection

Project in
Numerical Analysis for Machine Learning

Author: **Marine Legierski**

Student ID: 11153899
Advisor: Prof. Edie Miglio
Academic Year: 2025-26

Contents

Contents **i**

Introduction **1**

1 Convolution Neural Network **3**
1.1 Structure . . . 3
1.2 Convolution layer . . . 3
1.3 Pooling Layer . . . 5
1.4 Flatten Layer . . . 5
1.5 Activation function . . . 5
1.5.1 ReLU . . . 6
1.5.2 Softmax . . . 6
1.6 Filter hyperparameters . . . 7
1.6.1 Kernel size . . . 7
1.6.2 Padding . . . 7
1.6.3 Stride . . . 7
1.7 Tuning hyperparameters . . . 8
1.7.1 Parameter compatibility in convolution layer . . . 8
1.7.2 Complexity of the model . . . 9
1.8 Sources . . . 10

2 Binary classification for pneumonia detection **11**
2.1 Data Exploration . . . 11
2.1.1 Visualization . . . 11
2.2 Models evaluation . . . 13
2.2.1 Confusion matrix, Precision, Recall and F1-Score . . . 13
2.3 Models building . . . 14
2.3.1 Preprocessing . . . 14

2.3.2 Pipeline 14
2.3.3 Method 1 : basic CNN from scratch 14
2.3.4 Method 2 : Handling class imbalance 16
2.3.5 Method 3 : Data Augmentation 18
2.3.6 Method 4 : Use of Pre-trained models 22
2.4 Models comparison 25
2.5 Sources 26

Conclusion **27**

List of Figures **29**

List of Tables **31**

Introduction

This project focuses on the detection of pneumonia using lung X-ray medical images. This is a binary classification problem, where images are classified either as normal or pneumonia.

Pneumonia is a lung disease caused by an infection that inflames the air sacs in the lungs, which may fill with fluid and make breathing painful and difficult.

To handle this problem, we use deep learning methods, and more specifically Convolutional Neural Networks (CNN), which are well suited for image classification tasks. We use advanced techniques such as data augmentation and transfer learning, and we compare the different approaches in order to identify the best performing model.

This project is structured as follows. First, we introduce the architecture of Convolution Neural Network and the key concepts required to understand their structure. Then, this theoretical part is used to design and implement Convolution Neural Network models for this medical image classification task.

1 | Convolution Neural Network

1.1. Structure

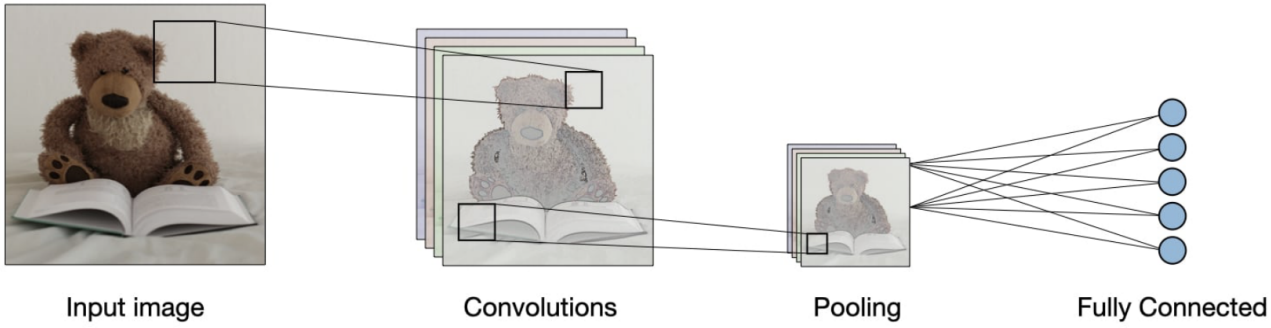


Figure 1.1: Structure of a CNN

The Convolutional Neural Network (CNN) is structured as illustrated in Figure 1.1. First, the input is an image, which can be either grayscale, with a single channel (as in our case with X-ray images), or color, with three channels (R, G, B). Convolutional kernels are then applied to extract the most relevant features from the image. Next, pooling layers are used to reduce the spatial dimensions of the feature maps. Finally, the resulting features are flattened and passed to fully connected layers, which produce the final class prediction. In the following sections, we will describe in detail the different layers that compose the CNN architecture.

1.2. Convolution layer

The convolutional layer (CONV) uses filters, also called kernels, to extract local features from the input image through convolution operations. Although the term convolution is commonly used in the context of CNN, the operation implemented in practice is not a true mathematical convolution as defined in signal processing.

Indeed, in signal processing, a convolution involves reversing the kernel before applying it to the input. In Convolutional Neural Networks, the kernel is not reversed, instead, it is directly superimposed onto the input. Therefore, the operation performed is more accurately described as correlation, which is defined as:

$$T[I](r,c) = \sum_{u,v} w(u,v) \ * I(r+u,\ c+v),$$

where I denotes the input image and w the kernel weights.

As illustrated in Figure 1.2, the feature map is obtained by computing an element-wise product between the kernel and the corresponding local region of the input, followed by a summation. For example, to obtain the value 19 in the output, we compute :

$$0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19.$$

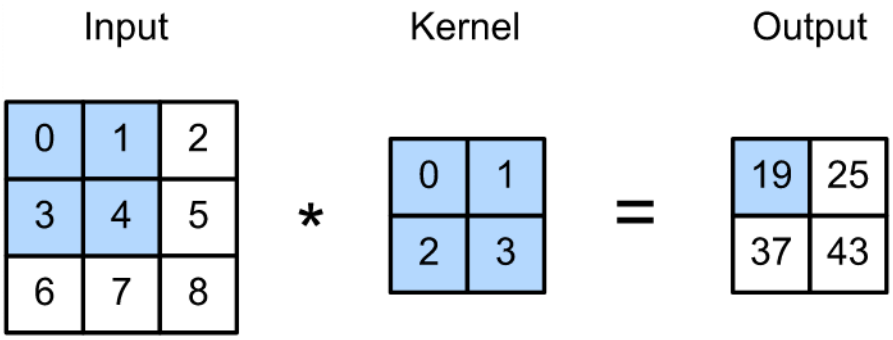


Figure 1.2: Correlation

Each filter has a predefined spatial size, for example 3×3, which is chosen before training. The purpose of these filters is to detect local patterns such as edges, lines, textures, or shapes. Multiple filters are used in parallel in order to extract different types of features from the input.

The values inside the kernels correspond to the model parameters, or weights, and are learned during training through backpropagation.

Thanks to the filters, this layer increases the number of channels while preserving, or almost preserving, the spatial dimensions of the input, depending on whether padding and stride are used or not.

1.3. Pooling Layer

After a convolutional layer, a pooling layer is generally applied to reduce the spatial dimensions of the feature maps while preserving the number of channels. This operation helps retain the most relevant features and significantly reduces the computational cost of the network. The two most common pooling strategies are Average pooling and Max pooling. In Average pooling, the output is computed as the mean value of the pixels within the pooling window, whereas in Max pooling, the maximum value within the window is selected. Figure 1.3 illustrates the Max pooling operation. In this example, a kernel of size 2×2 with a stride of 2 is used, resulting in a value of 20 for the pink region, since max(12,20,8,12)=20; the same reasoning is made for the other regions (yellow 30, purple 112 and green 37).

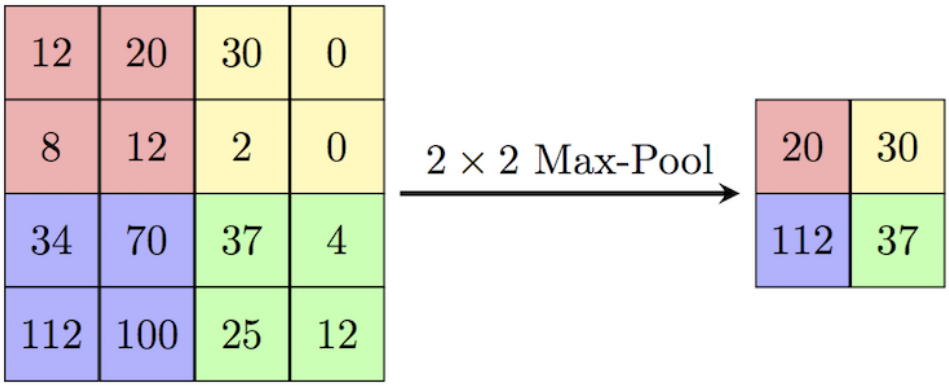


Figure 1.3: Max Pooling

1.4. Flatten Layer

This layer converts a three-dimensional tensor in the network into a one-dimensional vector so that it can be used as input to a fully connected layer for classification. For example, a tensor of size (3,24,24) is reshaped into a vector of length 3x24x24 = 1728.

1.5. Activation function

1.5.1. ReLU

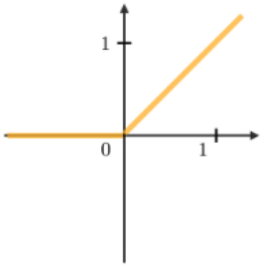


Figure 1.4: ReLU

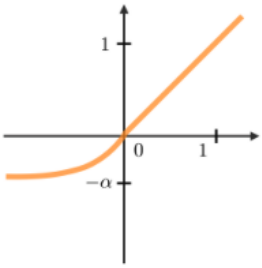


Figure 1.5: ELU

After the convolution layer, the Rectified Linear Unit (ReLU) is applied to the feature maps. This activation function introduces non linearity into the network, allowing the model to extract non linear patterns and complex shapes in the images. ReLU is defined as the maximum between 0 and the input value, meaning that all negative values are set to zero. While this property helps reduce vanishing gradient issues and speeds up training, ReLU is not differentiable at zero. For this reason, several variants of ReLU have been proposed, such as the Exponential Linear Unit (ELU), which aim to mitigate this limitation.

1.5.2. Softmax

The softmax function is applied at the final stage of the network to transform the output scores into interpretable values that can be interpreted as probabilities. Indeed, the outputs of the softmax function are constrained between 0 and 1, and their sum is equal to 1.

The softmax function is defined as :

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{C} e^{x_j}}$$

The main idea behind softmax is to use exponential function to preserve the relative ordering of the scores while amplifying differences between them. This ensures that higher scores correspond to higher probabilities and the resulting output forms a valid probability distribution.

In addition, the softmax function is preferred over the argmax function since it's differentiable, which allows gradients to be propagated during training.

1.6. Filter hyperparameters

1.6.1. Kernel size

Kernel size, also known as filter size, refers to the dimension of the sliding window over the input. It is an important hyperparamter that must be defined before training. Small kernel sizes are able to extract a much larger amount of information containing highly local features from the input. In contrast, larger kernel sizes capture more global information but reduce the spatial dimensions more quickly, which can lead to a loss of detail and, in practise, often results in poorer performance.

1.6.2. Padding

Padding is often necessary when the kernel extends beyond the boundaries of the activation map. It preserves information at the borders of the input, improves model performance and helps maintain the spatial dimensions of the features maps.

Without padding, information located at the edges of the input may be lost. The most common padding technique is zero padding, which consists of adding zeros around the borders of the input, as illustrated in Figure 1.6.

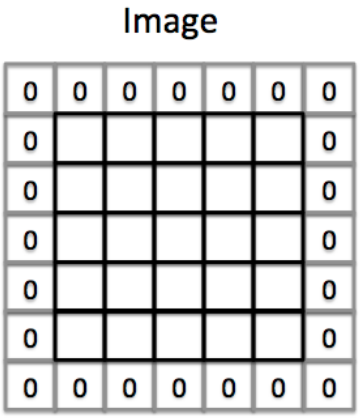


Figure 1.6: Zero padding

1.6.3. Stride

For a convolutional or a pooling operation, the stride denotes the number of pixels by which the window moves after each operation.

1.7. Tuning hyperparameters

1.7.1. Parameter compatibility in convolution layer

The spatial size of the feature maps obtained after applying a convolution to an input image can be computed using the formula shown in Figure 1.7. In this formula, I denotes the height/width of the input image, F the kernel size, P_{start} and P_{end} the padding applied to the input and S the stride. When zero padding is used, we typically have $P_{\text{start}} = P_{\text{end}} = 1$

As mentioned previously, the output size can remain unchanged depending on the chosen hyperparameters. For example, let's consider an input image of size 224x224, a kernel size of 3x3, padding such that $P_{\text{start}} = P_{\text{end}} = 1$ and a stride S = 1. The output size is then given by :

$$O = \frac{224 - 3 + 2 \times 1}{1} + 1 = 224$$

which means that the spatial resolution of the input is preserved.

However, if we increase the value of the stride, S = 2, while keeping the other parameters unchanged, the output size becomes :

$$O = \frac{224 - 3 + 2 \times 1}{2} + 1 = 112$$

In this case, the spatial resolution is reduced by a factor 2.

In conclusion, the spatial size of feature maps can be reduced either by increasing the stride in the convolutional layer or by using pooling operations.

$$O = \frac{I - F + P_{\text{start}} + P_{\text{end}}}{S} + 1$$

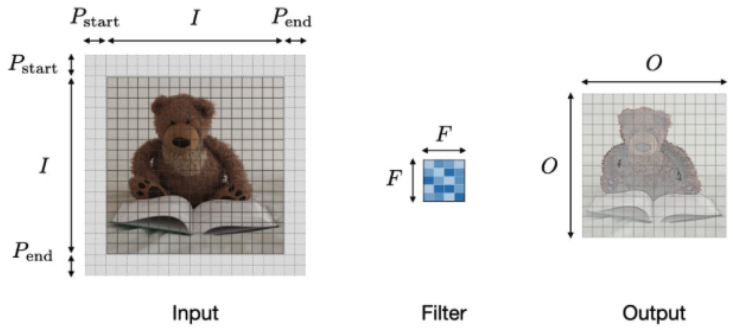


Figure 1.7: Size of the feature map after the convolution

1.7.2. Complexity of the model

To analyze the complexity of the model, it's often useful to determine the number of trainable parameters defined by its architecture. In a convolutional neural network, this can be computed layer by layer as follows :

- For a convolution layer, the number of trainable parameters depends on the kernel size (K_{h}, K_{w}), the number of input channels (C) and the number of filters (N_{f}), which corresponds to the number of output channels. The total number of trainable parameters in this layer is given by :

$$P = C \times K_h \times K_w \times N_f + N_f,$$

  where the additional N_{f} term accounts for the bias associated with each filter.

- For a pooling layer, there is no trainable parameter, since pooling operations perform fixed computations.

- For a fully connected layer, the number of trainable parameters depends on the number of input neurons N_{in} and the number of output neurons N_{out}. It's computed as :

$$P = N_{\text{in}} \times N_{\text{out}} + N_{\text{out}},$$

  where the additional term N_{out} term corresponds to the bias.

Therefore, when choosing hyperparameters such as kernel size, stride, padding and number of filters, it's important to keep in mind the model complexity to balance performance and computational cost.

1.8. Sources

Artificial Neural Networks and Deep Learning (AN2DL) POLIMI courses

https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks

https://poloclub.github.io/cnn-explainer/

2 | Binary classification for pneumonia detection

The second part of the project consists in applying the theoretical concepts presented in the first chapter to a practical case : the detection of pneumonia in chest X-ray images.

Our task is a binary classification problem, as there are two classes : normal and pneumonia. We use a Convolution Neural Network architecture to solve this problem. The dataset contains grayscale chest X-ray images, and the objective is to predict, for each image, whether the patient has a pneumonia or not.

Since this project is based on a Kaggle competition, the dataset is already split into training, validation and test sets. However, we reorganize the data splits in order to obtain a sufficiently large validation set for model selection and tuning.

2.1. Data Exploration

2.1.1. Visualization

Let us begin by visualizing a few examples from the dataset, Figure 2.1, comparing normal chest X-ray images with pneumonia cases. At first sight, the differences between the two classes are subtle. Pneumonia images may appear to contain larger bright regions in the lungs, possibly corresponding to reduced air content. However, since this visual assessment is subjective and we are not medical expert, we rely on deep learning methods to learn discriminative patterns directly from the data and solve this classification task.

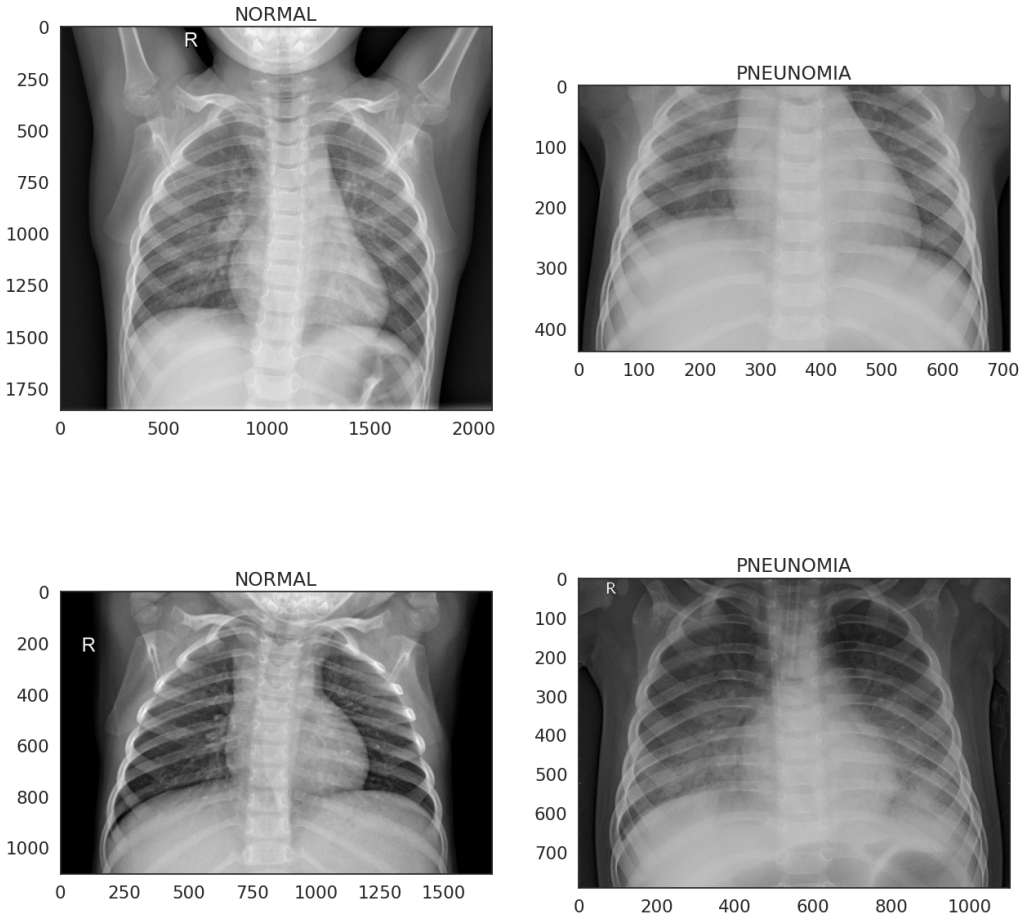


Figure 2.1: Visualization of chest images Normal VS Pneumonia

From this visualization, we also noticed that the X-ray images have different sizes, so they need to be resized during preprocessing. The images are also grayscale, which means that the input has only one channel.

Thanks to Table 2.1 below, we also observed that the dataset is imbalanced.

Repartition of Normal and Pneumonia through Train/Val/Test sets

	Normal	Pneumonia	Total
Train	1073	3099	4172
Val	268	776	1044
Test	234	390	624

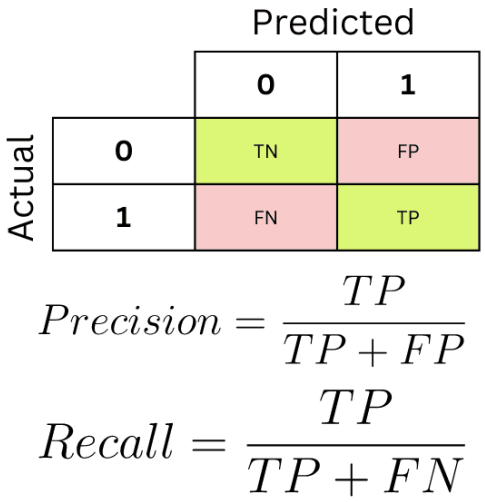
Table 2.1: Repartition of images in the dataset

In all data splits, pneumonia cases are more frequent than normal ones. To handle this imbalance, we used a stratified split between the training and the validation sets to keep the same class proportions. In these sets, pneumonia represents about 74% of the data. The test set contains around 60% pneumonia and 40% normal image, which helps evaluate whether the model can generalize instead of always predicting the most frequent class.

2.2. Models evaluation

In order to facilitate the interpretation of the results, we first introduce the evaluation metrics that we use to assess the models, before presenting their architectures.

2.2.1. Confusion matrix, Precision, Recall and F1-Score



$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Figure 2.2: Confusion matrix, Precision, Recall

To evaluate the performance of our model, we use the confusion matrix described above. In our case, class 0 corresponds to normal images and class 1 to pneumonia.

Using the confusion matrix, we can compute the following values :

- True negative (TN) : a normal case correctly predicted as normal
- True positive (TP) : a pneumonia case correctly predicted as pneumonia
- False negative (FN) : a pneumonia case incorrectly predicted as normal
- False positive (FP) : a normal case incorrectly predicted as pneumonia

Based on these four values, we compute evaluation metrics such as Precision, Recall (formula in Figure 2.2) and the F1-score. The F1-score combines Precision and Recall into a single metric. It is defined as :

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

In the medical context of this project, false negatives are more critical than false positives, since failing to detect pneumonia means that a sick patient will not receive appropriate treatment. For this reason, Recall is more important than Precision in our case, as it strongly penalizes models with a high number of false negatives.

2.3. Models building

2.3.1. Preprocessing

We now begin the preprocessing step to prepare data for model training. First, all images are resized to a fixed square resolution : 224x224. Then, the pixel values are scaled to the interval [0,1]. Since the training set contains more than 4 000 images, we don't use K-fold cross-validation. Instead, we apply a holdout validation strategy, in which the data are split only once into training and validation sets.

2.3.2. Pipeline

We iteratively develop several models with the objective of identifying the best performing one. We first train CNN from scratch and use different techniques to prevent overfitting. Then, we introduce some techniques to handle the class imbalance problem. Finally, we explore the use of data augmentation and pre-trained models through transfer learning.

2.3.3. Method 1 : basic CNN from scratch

For the first model, we create a CNN from scratch using a learning rate of 10^{-3}, a batch size of 64, and trained it for 100 epochs with early stopping using a patience of 10. The CNN architecture consists of two convolutional blocks with padding equal to 1 to preserve the edges and a pooling size of 2.

The structure of this first CNN is presented below in Figure 2.3, and the number of trainable parameters is computed as described in the previous chapter. This architecture contains a total number of 420 226 trainable parameters.

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1         [-1, 32, 224, 224]             320
              ReLU-2         [-1, 32, 224, 224]               0
         MaxPool2d-3         [-1, 32, 112, 112]               0
   VanillaCNNBlock-4         [-1, 32, 112, 112]               0
            Conv2d-5         [-1, 64, 112, 112]          18,496
              ReLU-6         [-1, 64, 112, 112]               0
         MaxPool2d-7           [-1, 64, 56, 56]               0
   VanillaCNNBlock-8           [-1, 64, 56, 56]               0
           Flatten-9               [-1, 200704]               0
          Dropout-10               [-1, 200704]               0
           Linear-11                    [-1, 2]         401,410
================================================================
Total params: 420,226
Trainable params: 420,226
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.19
Forward/backward pass size (MB): 49.00
Params size (MB): 1.60
Estimated Total Size (MB): 50.79
----------------------------------------------------------------
```

Figure 2.3: Structure of model 1 : CNN from scratch

By analyzing the training and validation losses and F1-scores shown in Figure 2.4, we observe that the model achieves very good performance on both datasets, with a 100% F1-score on the training set and a 97,9% F1-score on the validation set.The validation loss converges towards 0, with a slight increase at the end, which indicates mild overfitting. Therefore, for the next models, we introduce dropout and L2 regularization.

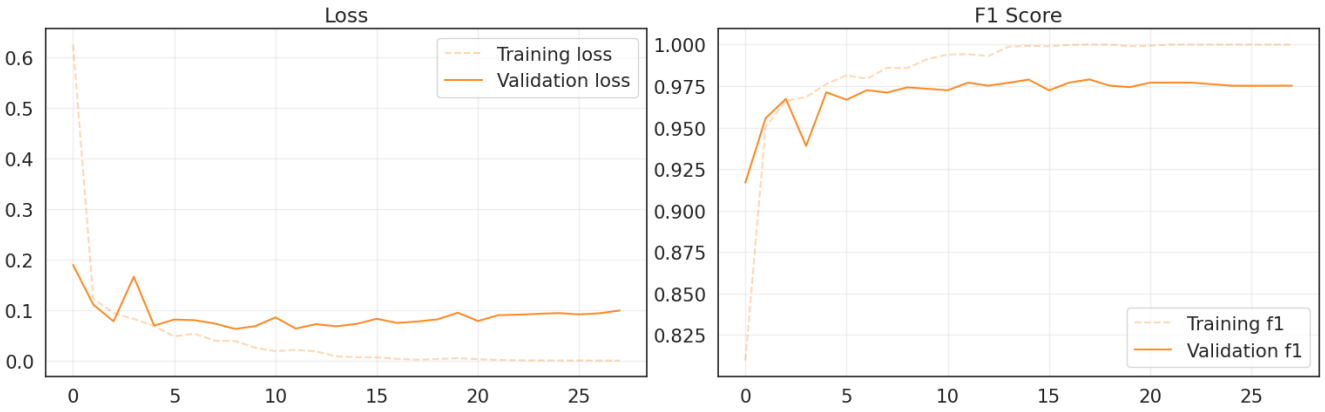


Figure 2.4: Loss and F1-score of model 1

The confusion matrix (Figure 2.5) confirms that the performance on the validation set is highly satisfactory, with both F1-score and Recall close to 98%.

However, on the test set, Figure 2.6, we observe that a large number of images are classified as pneumonia even when they are normal. This behavior is expected due to the strong class imbalance in the dataset, where pneumonia is the majority class. In particular, 166 false positives are observed. From a medical perspective, although false positives are preferred as false negatives; in our case they represent about 26% of the images, which can be costly especially if it would be necessary to have additional examinations.

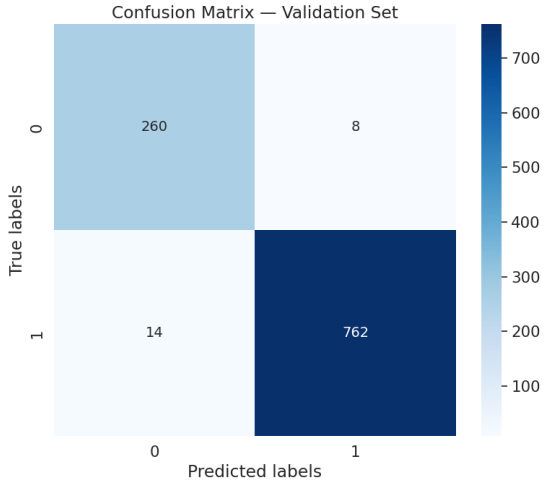


Figure 2.5: Confusion Matrix Val Set

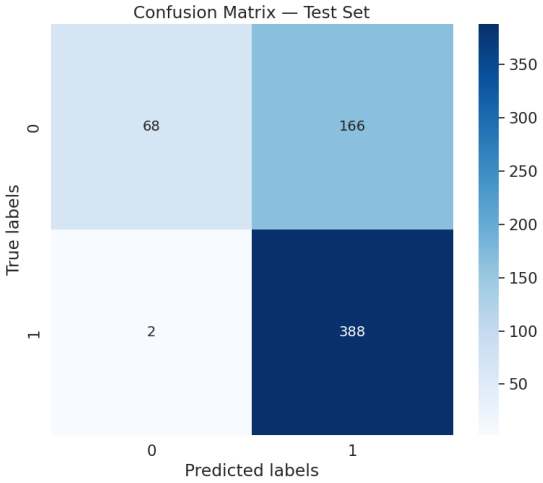


Figure 2.6: Confusion Matrix Test Set

Consequently, a different strategy is required to improve the classification performance on the test set due to class imbalance.

2.3.4. Method 2 : Handling class imbalance

Several techniques exist to handle class imbalance, such as undersampling the dataset to obtain an equal number of samples in each class or using a weighted loss to penalize the underrepresented class. We explore these techniques in practise in this section.

Undersampling

We first inverstigate the technique of undersampling, which consists of reducing the training dataset so that we have the same number of samples in each class, with 1073 normal and 1072 pneumonia images. Since the amount of training data is reduced, dropout and L2 regularization are introduced to limit overfitting. As in the previous method (1), the model shows good performance on both the training and validation sets. On the test set, a slight improvement is observed, as illustrated by the confusion matrix in Figure 2.7. While the number of false negatives remains unchanged at two, the number of false posi-

tives decreases to 145, corresponding to 21 fewer false positives compared to the previous model.

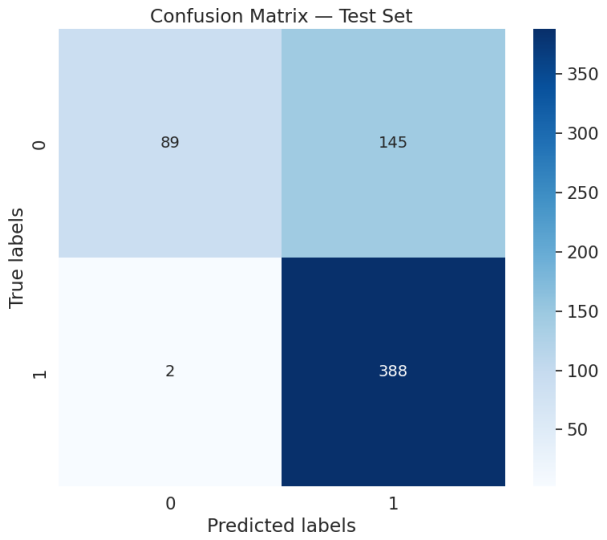


Figure 2.7: Confusion matrix on Test set (method : undersampling)

Weighted loss

Another approach to deal with class imbalance consists in assigning a higher weight to the underrepresented class (normal) in the loss function to penalize more strongly misclassification errors for this class. We consider a binary classification problem (K=2) with 1073 normal cases and 3099 pneumonia cases, leading to a total of N = 4172 samples. The class weights are computed as :

$$\text{weights} = \frac{N}{K \cdot \text{class_counts}}$$

where class_counts = [1 073, 3 099].

This results in a weight of 1,94 for the normal class and 0.67 for the pneumonia class.

Although class weighting is often used to handle class imbalance, it does not guarantee better generalization. In our case, even though the Recall increases, with only one false negative, the number of false positives also increases, leading to poorer performance on the test set, as illustrated in Figure 2.8.

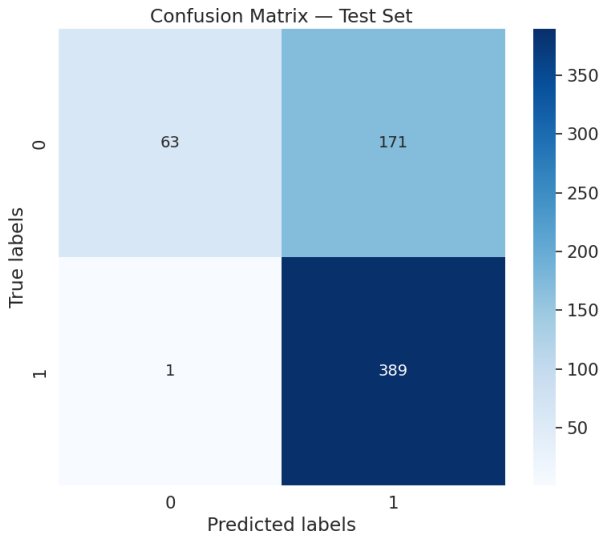


Figure 2.8: Confusion matrix on Test set (method : weighted loss)

2.3.5. Method 3 : Data Augmentation

Data Augmentation is a strategy used to improve the generalization of a model and potentially lead to better performance on the test set.

Two main families of data augmentation techniques can be distinguished. The first one consists of geometric transformations, such as random horizontal or vertical flips, rotations and translations.The second family includes photometric transformations, such as changes in brightness and contrast.

A key requirement for data augmentation is that the applied transformations must be label invariant, which means that they do not modify the class of the image. This aspect is particularly critical in the medical domain, where images are highly sensitive.

For instance, variations in brightness or contrast may play an important role in determining the presence of pneumonia, and modifying these properties could potentially change the semantic meaning of the image. For this reason, transformations are applied carefully.

We introduce the data augmentation techniques used in our experiments. Figure 2.9 shows the original image before any transformation, Figure 2.10 illustrates the geometric transformations while Figure 2.11, the photometric ones.

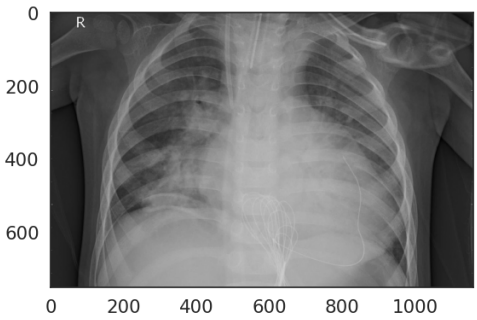
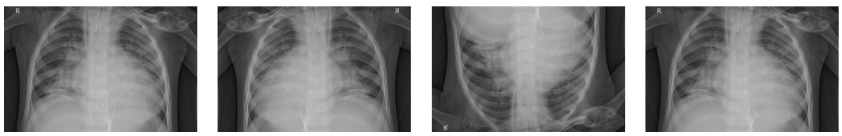
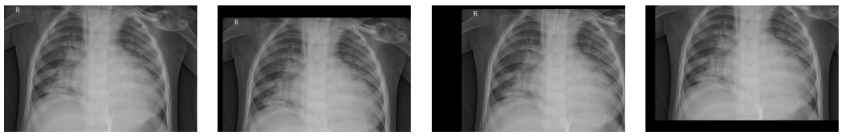


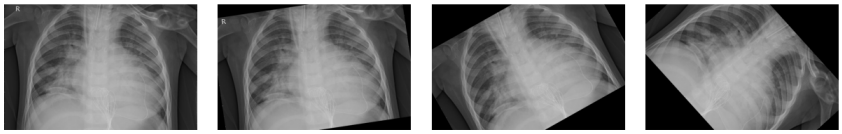
Figure 2.9: Original image before transformation



Random Flip (Horizontal / Vertical)

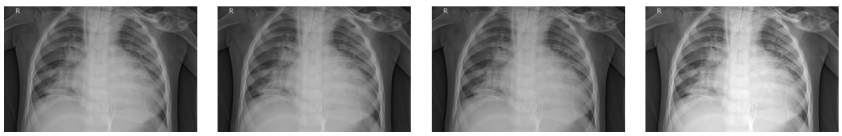


Translation

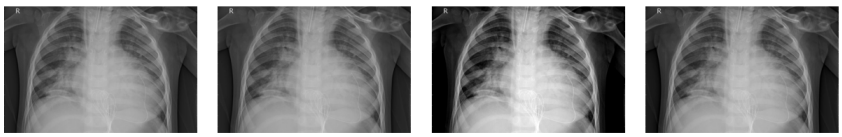


Rotation

Figure 2.10: Examples of geometric transformations



Brightness



Contrast

Figure 2.11: Examples of photometric transformations

First model : Random Flips

We train a first model using data augmentation, keeping the same hyperparameters as in the previous experiments. We add random horizontal and vertical flip with a probability of 0,5 in order to introduce variablility without excessively disturbing the original images.

The training and validation sets still demonstrate good performance, with a F1-score of 96,95% on the training set and 97,43% on the validation set. Moreover, compared to the previous methods, a slight improvement is observed on the test set, as illustrated by the confusion matrix in Figure 2.12, with 135 false positives. In particular, a F1-score of 84,24% is achieved, indicating an overall improvement in performance. However, the number of false negatives increases to eight, which reduces the Recall to 97,95%.

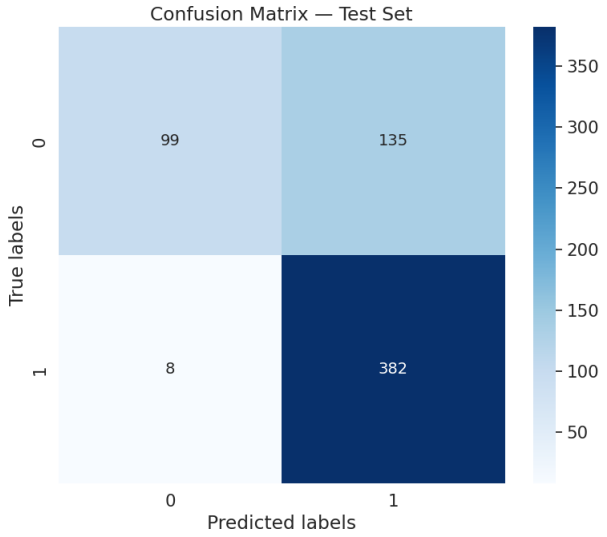


Figure 2.12: Confusion matrix on Test set (method : random flips)

Second model : Geometric Transformations

We train a second model using data augmentation similar to the flip based approach, but with the addition of translation and rotation transformations. Since we applied a larger number of transformations, the training process is significantly slower, requiring 38 epochs compared to 12 epochs for the previous model using only flips.

For this model, we observe that the loss decreases more slowly both on the training and validation sets, and the performance on these sets is worse than for the previous models, with a F1-score of approximatively 91% on both. However, on the test set, Figure 2.13, despite a non negligible number of false positives (111), the majority of images from the normal class are now correctly classified. As a result, the overall test performance

improves, reaching a F1-score of 85,22%.At the same time, the number of false negatives increases to 18 cases, which is undesirable in a medical context, where missing a pneumonia case is more critical than producing false positives.

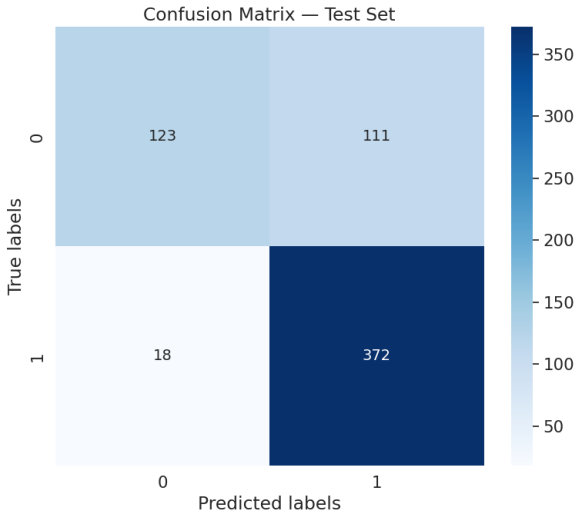


Figure 2.13: Confusion matrix on Test set (method : geometric transformations)

Third model : Photometric transformation

For the third model using data augmentation, we apply only the photometric techniques mentioned previously, brightness and contrast. As illustrated by the confusion matrix on the test set Figure 2.14, this approach does not perform well, as the majority of images are classified as pneumonia, leading to 173 normal images being misclassified as pneumonia. However, in terms of Recall, this model performs very well, with only one false negative.

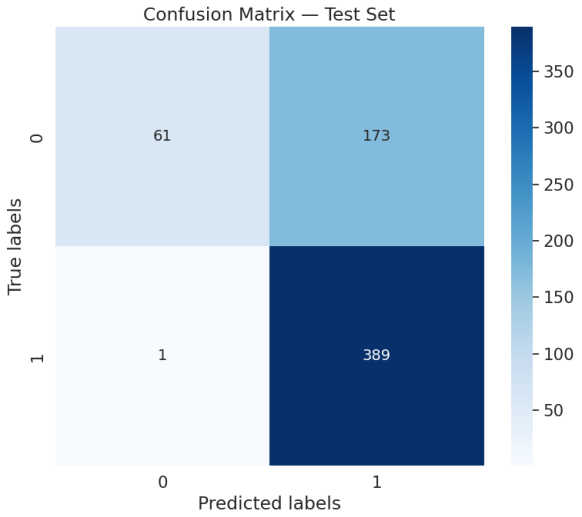


Figure 2.14: Confusion matrix on Test set (method : photometric transformations)

As mentioned previously, data augmentation is not always an appropriate solution, especially when the transformations significantly change the original images. In particular, modifying brightness and contrast in medical images can affect relevant features and potentially change the interpretation of the image.

It would be interesting to collaborate with a lung specialist to determine which transformations are invariant for these images.

2.3.6. Method 4 : Use of Pre-trained models

Transfer Learning

As illustrated in Figure 2.15, transfer learning is a technique that leverages pre-trained models, such as MobileNetV3 in this example. We use the pre-trained weights of the model to adapt it to our classification task. The only component that we modify is the classification head, where we adjust the number of outputs to fit our problem. Therefore, the classification head is configured with two output units corresponding to the normal and pneumonia classes. All backbone weights are frozen and only the multilayer perceptron classification head is trained.

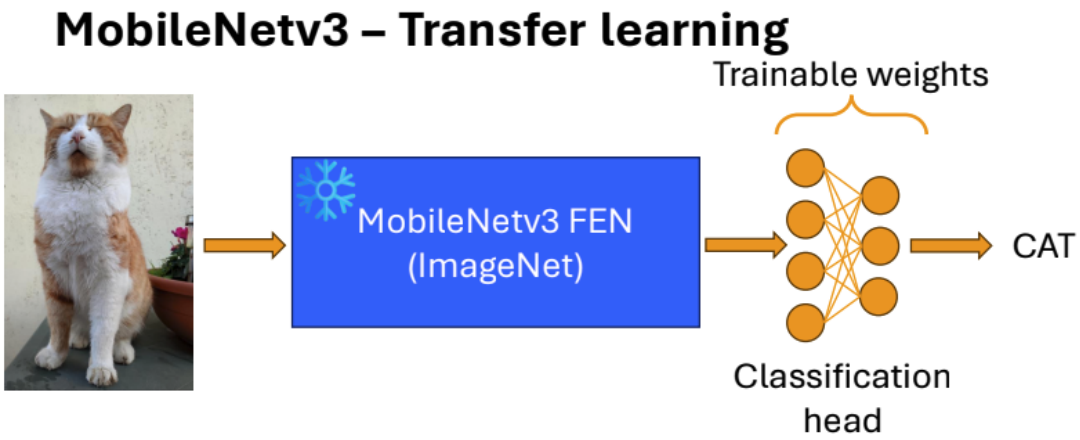


Figure 2.15: Transfer Learning

To apply transfer learning in our dataset, we chose as pre-trained model EfficientNet due to its good trade off beteween model complexity and performance.

In particular, EfficientNetB0 contains approximatively 5,3 million parameters, which is significantly fewer than traditional architectures such as ResNet, while achieving superior accuracy. Its strong transfer learning capabilities make it well suited for medical images classification tasks with limited data, such as pneumonia detection in our case.

Training a model using transfer learning requires significantly more time than training a model from scratch. In this model, we also apply random horizontal and vertical flips, as these transformations were shown to perform well in previous experiments. Overall, the results are quite good, as illustrated by the confusion matrix on the test set in Figure 2.16. In particular, we achieve a Recall of 98,72% with only 5 false negatives and a F1-score of 86,42% despite 116 false positives.

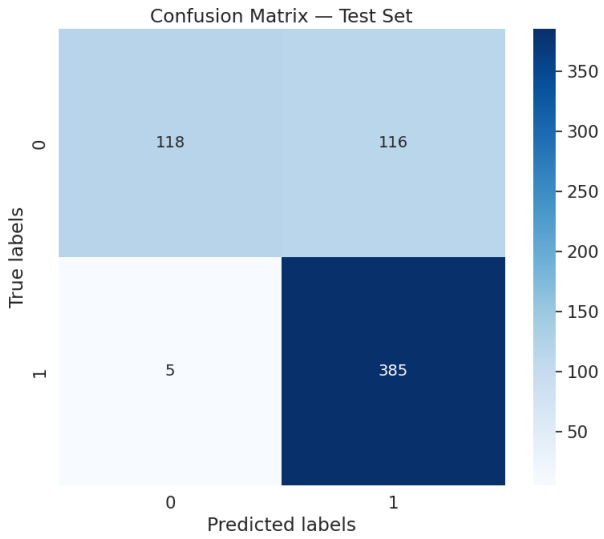


Figure 2.16: Confusion matrix on Test set (method : transfer learning)

Fine Tuning

As illustrated in Figure 2.17, fine tuning is another technique based on pre-trained models, similar to transfer learning. As in transfer learning, the output layer is adapted to match our classifcation task and the classification head is trained. In addition, while part of the backbone is frozen, some of its deeper layers are unfrozen and trained in order to better adapt the model to our specific task.

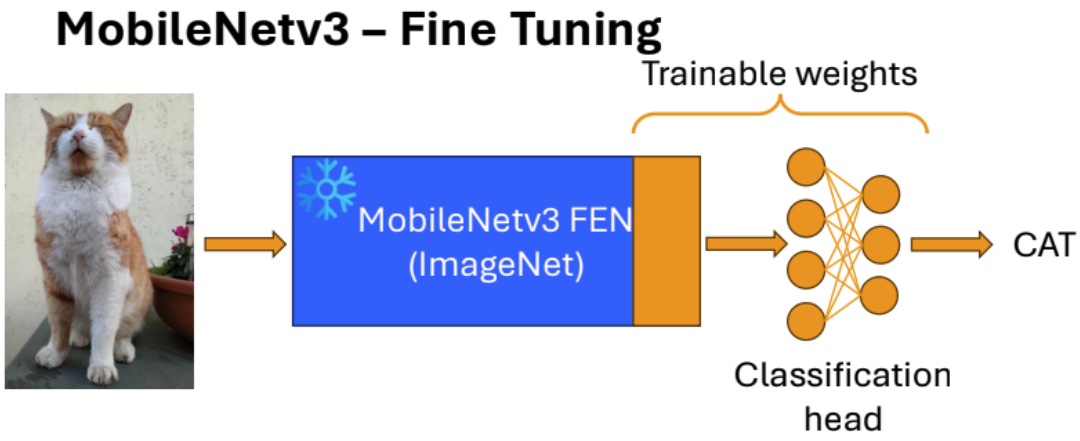


Figure 2.17: Fine tuning

We further apply EfficientNet using a fine-tuning strategy. We use a relatively small learning rate of 10^{-5} for the backbone and a learning rate of $5x10^{-4}$ for the classification head. However, the performance on the test set is worse than that obtained with transfer learning, as indicated by the confusion matrix in Figure 2.18, which shows the same number of false negatives but 133 false positives, corresponding to an increase of 17 false positives.

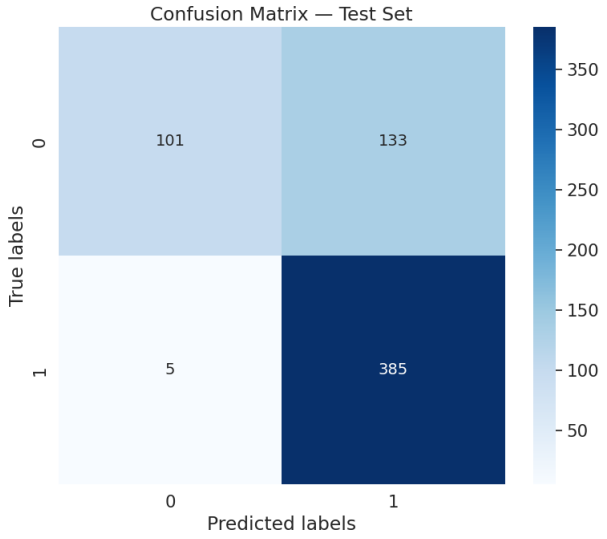


Figure 2.18: Confusion matrix on Test set (method : fine tuning)

2.4. Models comparison

Table 2.2: Comparison of models performance

Model	train_F1-score	val_F1-score	test_F1-score	test_Recall
CNN_from_scratch	100%	97,9%	82,2%	99,49%
CNN_undersampling	97,81%	97,39%	84,08%	99,49%
CNN_weighted_loss	100%	98,08%	81,89%	99,74%
CNN_augm_flips	96,95%	97,43%	84,24%	97,95%
CNN_augm_geometric	88,81%	91,46%	85,22%	95,38%
CNN_augm_photometric	97,41%	97,6%	81,72%	99,74%
Enet_transfer_learning	95,03%	96,36%	86,42%	98,72%
Enet_fine_tuning	97,48%	97,03%	84,8%	98,72%

Table 2.2 compares the performance of all the models developed in this project. The comparison mainly focuses on the F1-score on the test set and the Recall. As discussed previously, Recall is the most critical metric in the medical domain, since models that produce a large number of false negatives are particularly penalized.

The models that perform best in term of Recall are those using the weighted loss and the photometric data augmentation, both achieving a Recall of 99,74%. However, these two models show lower overall test performance, with the lowest test F1-scores, as they tend to predict a large number of pneumonia cases, resulting in many false positives.

Considering overall performance, the model based on EfficentNet with transfer learning achieves the best results, with a F1-score of 86,42%, followed by the model using the geometric transformations, which reaches a test F1-score of 85,22%.

In our point of view, the EfficientNet transfer learning model provides the best trade off between sensitivity and overall performance, as it achieves both the highest F1-score and a high Recall of 98,72%.

Finally, it would be valuable to discuss with a lung specialist to determine which model is most appropriate for this application and to further refine it in order to design an optimal solution.

2.5. Sources

LAB Polimi Artificial Neural Network and Deep Learning Courses

Conclusion

In conclusion, this project covers both the theoretical and practical aspects of Convolution Neural Networks.

In the first chapter, we present the theoretical foundations of CNN, including their architecture with the role of the different layers, and a particular emphasis on the correlation operation. We also discuss about the hyperparameters, such as kernel size and padding, and model complexity.

In the second chapter, we develop eight different models, starting from a CNN trained from scratch. Since the main challenge of our dataset is class imbalance, several strategies are explored to mitigate this issues, including undersampling and a weighted loss. To further improve generalization, we apply data augmentation techniques and explore pretrained models thanks to transfer learning and fine tuning. The different models are finally compared using evaluation metrics from the confusion matrix, such as Recall and F1-score.

Pneumonia detection proves to be particularly challenging, as it requires selecting a model that achieves both a high Recall, in order to minimize false negatives, and a strong overall performance, characterized by a high F1-score. Balancing these two objectives is essential in a medical context, where missing a pneumonia case can have serious consequences.

List of Figures

1.1 Structure of a CNN . . . 3
1.2 Correlation . . . 4
1.3 Max Pooling . . . 5
1.4 ReLU . . . 6
1.5 ELU . . . 6
1.6 Zero padding . . . 7
1.7 Size of the feature map after the convolution . . . 9

2.1 Visualization of chest images Normal VS Pneumonia . . . 12
2.2 Confusion matrix, Precision, Recall . . . 13
2.3 Structure of model 1 : CNN from scratch . . . 15
2.4 Loss and F1-score of model 1 . . . 15
2.5 Confusion Matrix Val Set . . . 16
2.6 Confusion Matrix Test Set . . . 16
2.7 Confusion matrix on Test set (method : undersampling) . . . 17
2.8 Confusion matrix on Test set (method : weighted loss) . . . 18
2.9 Original image before transformation . . . 19
2.10 Examples of geometric transformations . . . 19
2.11 Examples of photometric transformations . . . 19
2.12 Confusion matrix on Test set (method : random flips) . . . 20
2.13 Confusion matrix on Test set (method : geometric transformations) . . . 21
2.14 Confusion matrix on Test set (method : photometric transformations) . . . 21
2.15 Transfer Learning . . . 22
2.16 Confusion matrix on Test set (method : transfer learning) . . . 23
2.17 Fine tuning . . . 24
2.18 Confusion matrix on Test set (method : fine tuning) . . . 24

List of Tables

2.1 Repartition of images in the dataset 12
2.2 Comparison of models performance 25